
Stable Blockwise Attention under Extreme Scores: Numerical Fidelity and the Boundary of Local Masking

Abstract

Does computing attention in blocks preserve global attention when scores are extreme, and does restricting attention to a local window have the same effect? We study these distinct transformations with an independent CPU NumPy implementation. A running maximum and rescaled denominator and value numerator retain the chosen masked attention operator in real arithmetic. Across a fixed sweep of 648 blockwise comparisons, every output is finite and meets the prospective absolute-error tolerance against a stable dense float64 reference. The maximum discrepancies are 3.53×10^{-13} for float64 and 2.99×10^{-5} for float32. Direct exponentiation without maximum subtraction produces nonfinite outputs in all 108 high-scale cases, compared with none of 108 ordinary-scale cases. In a separate, exactly specified example, replacing global attention with a radius-2 local mask changes the first query output from 1 to 0. These results distinguish arithmetic stabilization, storage tiling, and a change of operator. They support numerical fidelity on the tested configurations, not a universal finite-precision bound, a GPU performance result, or a claim about downstream model quality.

1 Introduction

Attention combines values using weights normalized across a query’s permitted keys. Consequently, two superficially similar ways to reduce the size of an intermediate score matrix have different obligations. A tiled implementation must preserve contributions from every permitted key, including contributions processed earlier. A local attention rule instead removes some keys from the permitted set and renormalizes the remaining weights. The first can preserve the original operator; the second need not.

Scaled dot-product attention defines the normalization target [Vaswani et al., 2017]. FlashAttention demonstrates how blockwise normalization can support an exact attention algorithm, with IO analysis and GPU implementation forming additional parts of that work [Dao et al., 2022]. This working paper isolates a narrower question: whether a small independent implementation of the normalization mechanism remains numerically faithful under a fixed extreme-score stress test, and what a local-mask counterexample reveals about the meaning of “exact.” The stable merge is established prior work, not a new attention algorithm.

Our contribution is a bounded reproduction and a separation of failure mechanisms. We give the row-wise invariant and its empty-tile conditions, compare a blockwise implementation with a separately organized dense reference, retain all unstable-comparator outcomes, and examine a construction in which discarded distant information changes the answer. The evidence addresses forward attention on generated arrays. It does not test a trained Transformer or reproduce FlashAttention’s runtime results. This scope makes it possible to answer the operator question directly without treating numerical agreement as evidence of speed or model quality.

2 Operator and stable blockwise computation

2.1 The target includes its mask

Let $Q, K \in \mathbb{R}^{N \times d}$ and $V \in \mathbb{R}^{N \times d_v}$, with rows q_i, k_j, v_j . A Boolean mask M defines the allowed set $A_i = \{j : M_{ij} = 1\}$. For finite dot products and $A_i \neq \emptyset$, define

$$s_{ij} = q_i^\top k_j / \sqrt{d}, \quad o_i = \frac{\sum_{j \in A_i} \exp(s_{ij} - \mu_i) v_j}{\sum_{j \in A_i} \exp(s_{ij} - \mu_i)}, \quad \mu_i = \max_{j \in A_i} s_{ij}. \quad (1)$$

Subtracting μ_i cancels between numerator and denominator in real arithmetic and makes the allowed exponential arguments nonpositive. “Global” below means that all keys permitted by the specified mask are considered, rather than only keys within a processing tile. The unmasked case has $A_i = \{0, \dots, N-1\}$; causal and random masks define other targets. Exact evaluation of a causal target is not evaluation of unmasked attention.

A fully masked row has no normalized distribution under Equation (1). We choose rejection rather than assigning an arbitrary zero output. This contract is separate from a temporarily empty tile: a query can have no allowed key in the current tile while still having a valid target over the full sequence.

2.2 Rescaling preserves earlier contributions

For one query, process disjoint key tiles in sequence. After a processed allowed set $S \neq \emptyset$, maintain

$$m = \max_{j \in S} s_{ij}, \quad \ell = \sum_{j \in S} e^{s_{ij} - m}, \quad a = \sum_{j \in S} e^{s_{ij} - m} v_j. \quad (2)$$

For the new allowed subset B , take its maximum to be $-\infty$ if empty. With $m' = \max(m, \max_{j \in B} s_{ij})$ and finite m' , update

$$\begin{aligned} \alpha &= e^{m - m'}, & p_j &= e^{s_{ij} - m'} \quad (j \in B), \\ \ell' &= \alpha \ell + \sum_{j \in B} p_j, & a' &= \alpha a + \sum_{j \in B} p_j v_j. \end{aligned} \quad (3)$$

The crucial factor is α : when the maximum increases, it expresses all earlier contributions in the new exponential scale. Independently normalizing each tile and averaging its output would discard the relative normalization masses.

Correctness in real arithmetic. Initialize $m = -\infty$, $\ell = 0$, and $a = 0$. The first nonempty tile establishes Equation (2). If the invariant holds for S , multiplying its sums by $e^{m - m'}$ replaces each factor $e^{s_{ij} - m}$ by $e^{s_{ij} - m'}$. Adding the new tile therefore gives exactly the sums over $S \cup B$, while m' is its maximum. Induction establishes the invariant after all tiles. If every allowed key is visited exactly once, final division a/ℓ equals Equation (1). This is the same normalization principle used in FlashAttention’s tiled construction [Dao et al., 2022, Section 3.1]; the induction here specifies the masked row and unnormalized-numerator form used in our implementation.

Empty tiles and numerical conditions. If both the processed set and current tile are empty, evaluating $-\infty - (-\infty)$ would create an invalid value. The implementation substitutes zero as a temporary subtraction origin when m' is not finite. Masked scores are $-\infty$, so their exponentials and the empty-state rescaling become zero; ℓ and a remain zero and m remains $-\infty$. An empty tile after a finite state has $\alpha = 1$ and adds zero. Any fully masked row is rejected before the loop, so final division is reached only for rows with an allowed key. This guard assumes finite allowed scores; it is not a general treatment of overflowing dot products or NaN inputs.

The algebra does not imply bitwise equality. Floating-point score computation, summation order, rescaling, and final division introduce rounding, and very small shifted exponentials can underflow. Maximum subtraction prevents overflow in the exponentiation step for finite scores; it does not prevent overflow while forming the scores or guarantee arbitrarily small output error.

Table 1: Fixed numerical sweep. Every Cartesian-product configuration is retained; the three block sizes share each dense reference.

Factor	Values or definition
Sequence length N	7, 31, 128
Key/query dimension d ; value dimension d_v	8, 32; 5
Seed index; Q/K scale c	0, 1, 2; 1, 100
Mask	Unmasked, causal ($j \leq i$), random nonempty
Input and blockwise accumulator dtype	float64, float32
Block size b	1, 7, 32
Acceptance rule	Finite output and max absolute error $\leq \tau$
Prospective tolerance τ	10^{-10} (float64); 5×10^{-4} (float32)

3 Experimental design

The experiment uses independent NumPy functions for stable dense and blockwise forward attention. The dense reference forms the full score matrix in float64, replaces forbidden scores with $-\infty$, subtracts each row maximum, normalizes once, and multiplies by float64 values. The blockwise function uses query and key blocks of the same nominal size b , computes scores and accumulators in the input dtype, and divides the accumulated numerator by its denominator after all key tiles. Edge tiles are shortened to the available rows or columns.

Table 1 specifies $3 \cdot 3 \cdot 2 \cdot 2 \cdot 3 \cdot 2 \cdot 3 = 648$ comparisons. For each seed index r , length, and dimension, the NumPy generator seed is $100000r + 100N + d$. Standard normal draws generate Q/c , K/c , and V ; the generator restarts with the same seed at each scale. Thus scale 100 changes both Q and K and multiplies the corresponding real-arithmetic scores by 10^4 , while preserving V and the random mask. The mask is drawn with entry condition uniform random value > 0.5 , then its diagonal is forced true. Causal masks also keep the diagonal. These conditions enforce nonempty rows, exercise masked tiles, and include block boundaries that do not evenly divide the sequence length.

Inputs are cast to the tested dtype before either implementation runs. In particular, the float32 reference promotes the already quantized inputs to float64; it does not compare against the original pre-quantization samples. For each configuration we record

$$E = \max_{i,t} |\hat{O}_{it} - O_{it}^{\text{dense64}}|, \quad (4)$$

where t indexes value components. The reference is a higher-precision numerical computation, not an arbitrary-precision oracle. The tolerances were fixed before execution; they are acceptance thresholds for this study, not statistical confidence limits.

A diagnostic comparator computes dense exponentials without maximum subtraction. It has 216 cases because it does not depend on block size. A failure means at least one nonfinite output entry. Separately, an all-false 16×16 mask checks explicit rejection, and a fixed far-key example tests semantic equivalence of local and global masks. The recorded environment is Python 3.12.14, NumPy 2.3.5, Linux x86_64, with OpenBLAS, OMP, and MKL thread variables set to 1. There are no timing observations. Configuration errors are deterministic outcomes of this sweep; the shared inputs, masks, and block variants are not independent statistical replicates, so we report counts and maxima without confidence intervals.

4 Results

4.1 Stable blockwise outputs meet every fixed tolerance

All 648 comparisons produce finite outputs and pass the respective threshold. Table 2 gives maxima by dtype and scale; Figure 1 retains the distribution of positive discrepancies and explicitly counts zeros. The overall maxima are $3.532729664357248 \times 10^{-13}$ for float64 and $2.988110650203879 \times 10^{-5}$ for float32. Both occur with seed index 2, $N = 128$, $d = 32$, scale 100, and the random nonempty mask; the maximizing blocks are 1 and 7, respectively.

Table 2: Maximum absolute error against the dense float64 reference. Each row contains 162 comparisons; all 162 are finite and pass. Zeros mean exact equality of the computed output arrays for that configuration, not mathematical exactness of every operation.

Dtype	Q/K scale	Maximum E	Tolerance	Zero-error cases
float64	1	1.22125×10^{-15}	10^{-10}	0
float64	100	3.53273×10^{-13}	10^{-10}	129
float32	1	5.33957×10^{-7}	5×10^{-4}	0
float32	100	2.98811×10^{-5}	5×10^{-4}	105

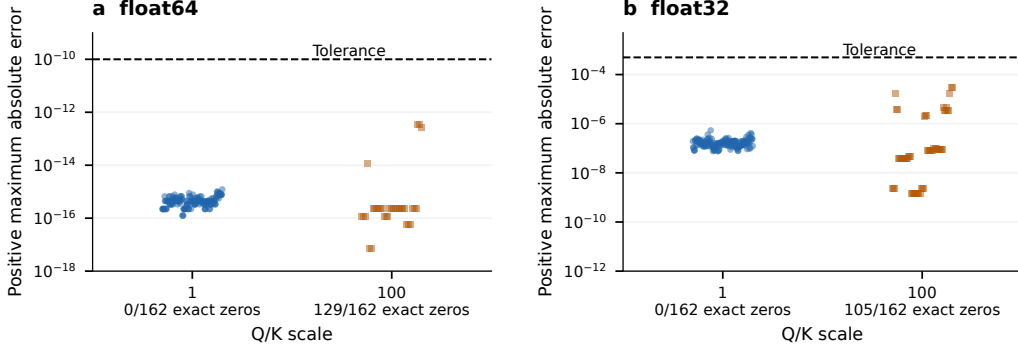


Figure 1: Output discrepancy over the fixed sweep: 324 configurations per dtype and 162 per scale. All positive maximum errors are plotted; exact zeros are counted under each group and are not assigned an artificial log floor. Dashed lines mark prospective tolerances. The panels use different labeled vertical ranges. Horizontal jitter separates points and has no quantitative meaning. These are configuration outcomes, not independent replicates.

The high-scale group simultaneously has larger worst-case error and many exact-zero discrepancies. These are compatible observations: agreement on some concentrated distributions does not bound sensitivity on others. Concentration of softmax weights is a possible explanation for the zeros, but this experiment does not record a concentration metric and cannot assign that cause to each case. The numerical finding is therefore deliberately limited to the observed errors, rather than a claim that increasing score magnitude always improves or worsens fidelity.

4.2 Unstable exponentiation fails despite a valid target

The unstabilized comparator produces nonfinite outputs in 108 of 216 cases. All 108 scale-100 cases fail, while none of the 108 scale-1 cases does (Figure 2a). This comparison isolates the consequence of omitting maximum subtraction in these inputs. It is not a comparison against a competitive stable dense implementation: stable dense attention supplies the reference throughout. The count is per configuration, not the number of nonfinite scalar entries.

Both dense and blockwise functions raise `ValueError` with the message “fully masked row” on the all-false mask. That test records rejection of an entirely masked matrix; the implementation’s rowwise precheck also defines rejection when any row is empty. No separate partially empty-row test is claimed. For the valid causal and random masks, finite results throughout the sweep are consistent with correct handling of empty tiles and later allowed contributions.

4.3 A local mask changes the answer even without numerical stress

Set $N = 16$, $d = 8$, $Q = K = 0$, and scalar values $v_{15} = 16$ with all other $v_j = 0$. Under the full noncausal mask all scores are zero and all 16 weights are $1/16$, so every output is 1. Replace that

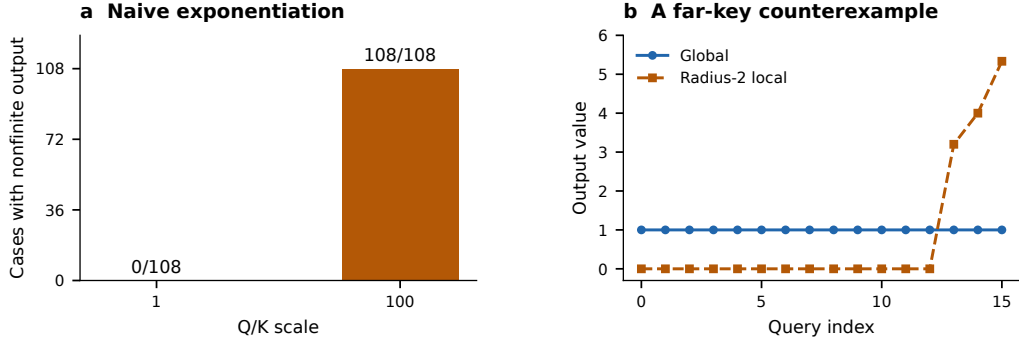


Figure 2: Two distinct boundaries. (a) Direct dense exponentiation without maximum subtraction fails in all 108 high-scale cases and none of 108 ordinary-scale cases; both Q and K are scaled. (b) A noncausal radius-2 mask changes the result for $N = 16$, zero Q and K, and only the last value equal to 16. The global output is 1 everywhere; local output at query 0 is 0 and at query 15 is $16/3$. Lines connect discrete query indices for readability. Neither panel measures runtime or downstream accuracy.

mask with $M_{ij}^{\text{local}} = 1$ exactly when $|i - j| \leq 2$. Query 0 averages three zeros: its local output is 0 instead of 1. This discrepancy occurs without extreme exponentials or a change of summation order.

The entire measured output vector reveals the renormalization effect (Figure 2b). Local outputs are zero at queries 0 through 12, then $16/5 = 3.2$, $16/4 = 4$, and $16/3$ at queries 13, 14, and 15. The maximum absolute discrepancy from global attention is consequently $13/3 \approx 4.333333333333333$. Including the distant nonzero value in the last windows raises its weight because those windows have fewer allowed keys. This is a counterexample to equality of operators, not evidence that every local attention model performs poorly on its task.

5 Interpretation, related work, and limitations

Stable tiling and local masking answer different questions. Equations (2)–(3) change the evaluation order while keeping the allowed set fixed. Section 4.3 changes that set. Stable blockwise evaluation can also compute a local target exactly in real arithmetic; it cannot restore values excluded by the local target. Conversely, a numerically stable implementation can faithfully compute an operator that is unsuitable for a particular modeling objective. Fidelity of computation and suitability of the model require different evidence.

Relation to prior attention mechanisms. The operator in Equation (1) is a masked form of scaled dot-product attention [Vaswani et al., 2017, Section 3.2]. FlashAttention’s Section 3.1 develops tiled normalization, while its sparse extension explicitly changes the attention pattern [Dao et al., 2022]. Our derivation and tests reproduce a normalization mechanism independently, with arbitrary Boolean masks and a separate value dimension, rather than proposing that mechanism anew. The empirical increment is the specified small CPU stress sweep and the explicit semantic counterexample. The experiment is not a broad survey or comparative benchmark of efficient attention methods.

Score-tile size is not peak memory. A block has at most $\min(N, b)^2$ score elements. At $N = 128, b = 32$, this is 1,024 rather than 16,384 elements in a dense score matrix, a factor of 16 in this one array shape. This is exact allocation-shape accounting, not a memory measurement. The implementation still allocates an $N \times N$ Boolean mask, retains inputs and outputs, and uses additional temporaries. It therefore does not establish linear total memory, reduced resident memory, reduced hardware traffic, or a speedup. Visiting all key tiles retains quadratic pairwise work in the unmasked case.

Boundary of the source reproduction. The pinned FlashAttention v1.0.9 interface imports Torch and a CUDA extension, exposes causal masking and dropout, and dispatches forward and backward operations [FlashAttention contributors]. Its documented unpadding arrays include a head axis and cumulative sequence lengths. This study instead uses single-head NumPy matrices and a generic value dimension of 5; it neither implements that interface nor executes its kernels. The public code is an interface reference, not the source of our measured outputs.

Unmeasured regimes. The longest tested sequence is 128, only three seed indices are used, and the random distributions are prescribed rather than learned activations. The sweep does not establish error bounds for longer sequences, nearly tied extreme scores, other mask distributions, large value magnitudes, adversarial rounding cases, or dot-product overflow. Relative error and arbitrary-precision references were not evaluated. Half precision, backward gradients, dropout, training, GPU kernels, throughput, memory traffic, and downstream accuracy remain untested. A next numerical study could vary score gaps and value magnitudes while checking against higher precision; a performance study would need actual timing and memory measurements of scope-matched implementations. Those are distinct extensions, not outcomes of the present experiment.

6 Conclusion

Stable blockwise normalization preserves attention over the same allowed keys in real arithmetic and stays within the declared numerical tolerances in all 648 tested CPU configurations, including the high-score group where direct exponentiation fails. A local mask has a different effect: the far-key example changes an output from 1 to 0 even when every score is zero. The supported distinction is between a stable evaluation of a fixed operator and a change to the operator itself. The numerical sweep substantiates the former on its tested domain; the counterexample rules out general equivalence of the latter with global attention.

References

- Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. *arXiv preprint arXiv:2205.14135v2*, 2022. URL <https://arxiv.org/pdf/2205.14135v2>. Inspected version dated June 24, 2022; 34 pages.
- FlashAttention contributors. FlashAttention python interface, tag v1.0.9. Source code, Git blob 07e97ce96f688bd62446b432005e2b87a1739aab. URL https://github.com/Dao-AI-Lab/flash-attention/blob/v1.0.9/flash_attn/flash_attn_interface.py. Inspected October 6, 2026.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017. URL https://papers.nips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf.